

README

Live System Topology - Docker Compose

ภาพรวมโครงการ (Project Overview)

ระบบ Live System Topology เป็นแอปพลิเคชันสำหรับการแสดงภาพ **โครงข่ายความสัมพันธ์ (relationship topology)** แบบ Real-time ด้วยชุดเทคโนโลยีสมัยใหม่ที่ทำงานภายในคอนเทนเนอร์ Docker

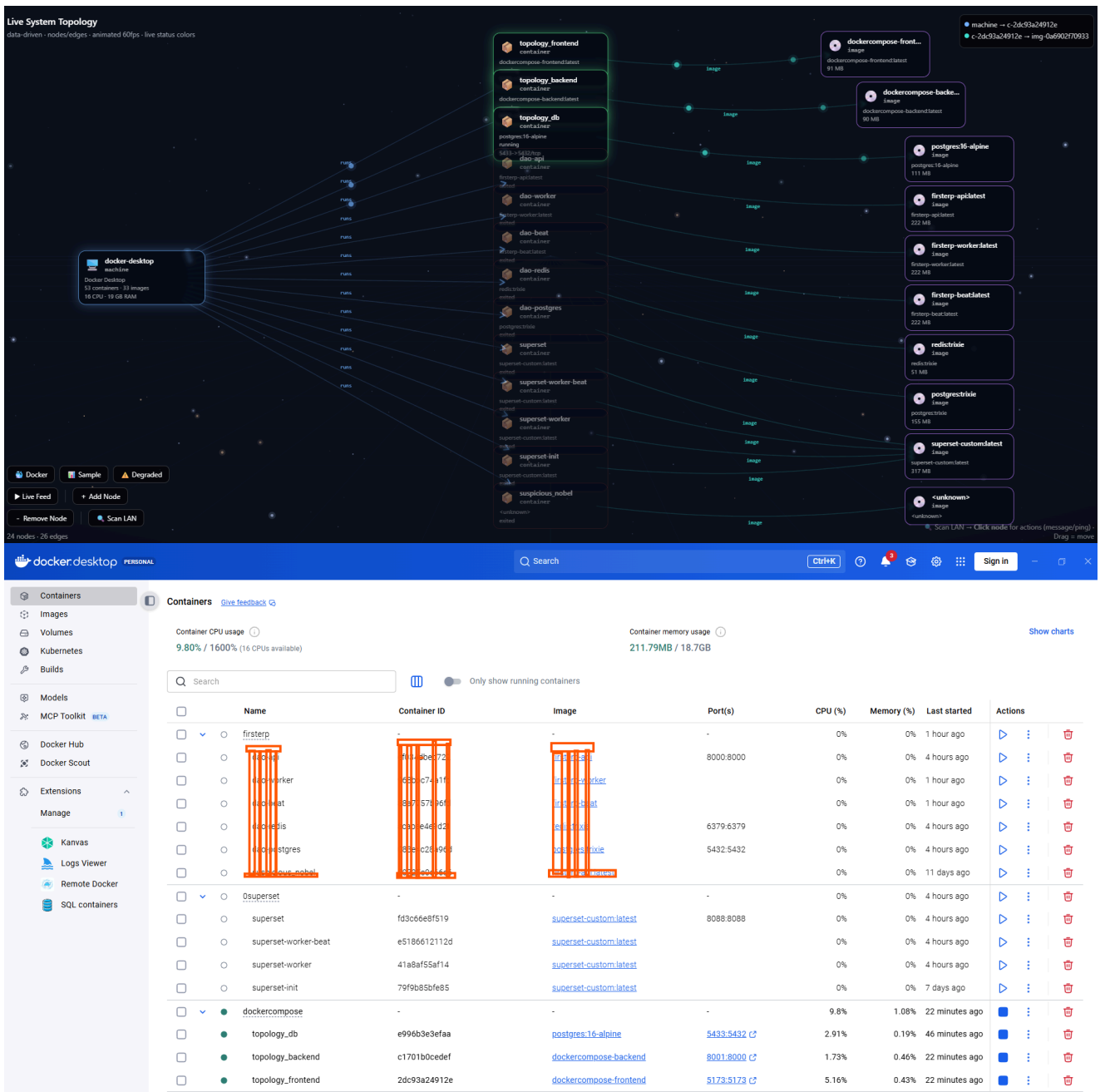
ระบบรองรับ 2 มุมมองหลัก:

1. 🐳 **Docker Topology (ค่าเริ่มต้น)** — อ่านข้อมูลจริงจาก Docker daemon แล้ววาดเป็นโครงข่าย 3 ชั้น **Machine → Container → Image**
2. 🔍 **LAN Scan** — ค้นหาอุปกรณ์บนเครือข่ายด้วย ping sweep / ARP scanning

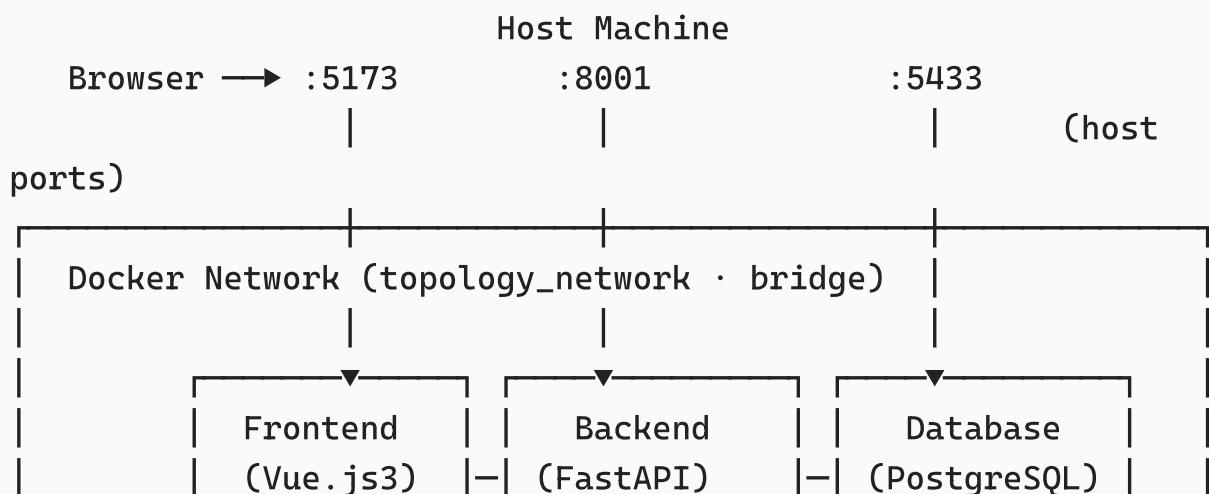
🎯 วัตถุประสงค์

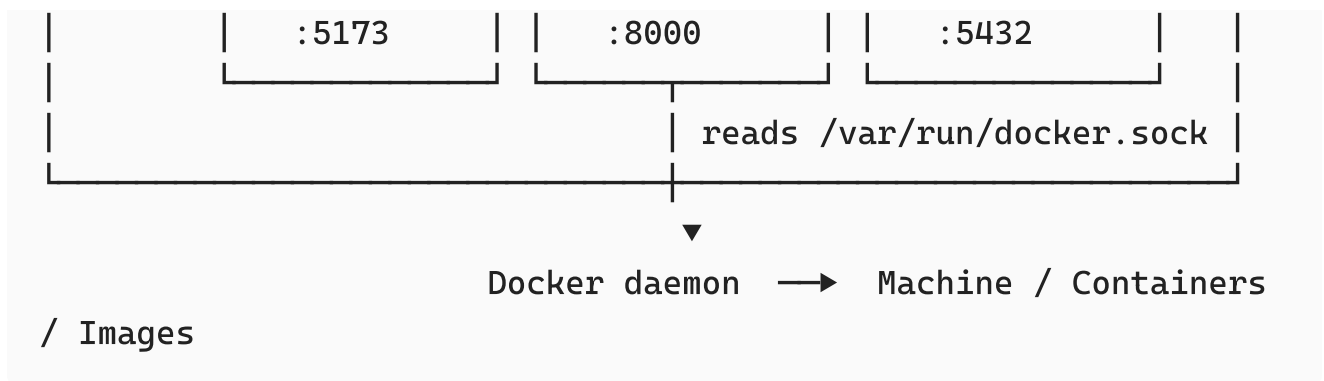
- 🐳 **มองเห็นความสัมพันธ์ของ Docker** - แสดง Machine, Container และ Image ที่เชื่อมโยงกันเป็นกราฟ
- ✨ **ค้นหาอุปกรณ์บนเครือข่าย** - Network discovery โดยใช้ ping sweep และ ARP scanning
- 📄 **สื่อสารกับเครื่องบนเครือข่าย** - ส่งข้อความและคำสั่งไปยังอุปกรณ์
- 📊 **แสดงสถานะแบบ Real-time** - ติดตามสถานะของแต่ละ node (สี เขียว/เหลือง/แดง)
- 🐳 **Microservices Architecture** - 3 containers สำหรับ Frontend, Backend, Database

Result



สถาปัตยกรรม (Architecture)





หมายเหตุพอร์ต: ภายใน Docker network คอนเทนเนอร์คุยกันที่ 8000 / 5432 ตามเดิม แต่พอร์ตที่ map ออกมาที่เครื่อง host เปลี่ยนเป็น 8001 (backend) และ 5433 (database) เพื่อเลี่ยงการชนกับบริการอื่นบนเครื่อง

Containers

1. Frontend Container (Vue.js 3 + Tailwind CSS)

- **Image:** Node.js 20-Alpine
- **Port:** 5173
- **Tech Stack:**
 - Vue.js 3 (Composition API)
 - Vite (Build tool)
 - Tailwind CSS (Styling)
 - Axios (HTTP client)
- **Features:**
 - Topology visualization with Canvas
 - Real-time node/edge rendering
 - Network scanning UI
 - Device action panel (send message, ping, wake)

2. Backend Container (Python FastAPI)

- **Image:** Python 3.11-Slim
- **Port:** 8000 ภายใน container → map ออกที่ host เป็น 8001
- **Volume พิเศษ:** /var/run/docker.sock:/var/run/docker.sock:ro เพื่ออ่านข้อมูล Docker daemon
- **Tech Stack:**
 - FastAPI (Web framework)

- SQLAlchemy (ORM)
- psycopg2 (PostgreSQL adapter)
- Docker SDK for Python (อ่าน Machine/Container/Image)
- Scapy (Network scanning)
- **APIs:**
 - GET /api/topology - อ่านโครงสร้าง Docker (Machine → Container → Image)
 - GET /api/hosts - Discover network hosts (LAN scan)
 - POST /api/action - Execute actions (message, ping)
 - GET /api/history - Get action history
 - GET /api/hosts/{ip} - Get specific host details

3. Database Container (PostgreSQL)

- **Image:** PostgreSQL 16-Alpine
- **Port:** 5432 ภายใน container → map ออกที่ host เป็น **5433**
- **Database:** topology_db
- **Tables:**
 - hosts - Network device information
 - scan_results - Historical scan data
 - action_history - Action logs
 - messages - Message history



Quick Start

Prerequisites

- Docker & Docker Compose
- Git

Installation & Running

1. Clone/Copy Project

```
\CT519 - การประมวลผลคลาวด์\Homework\Docker compose"
```

2. Build & Start Containers

```
# Build images and start services
```

```
docker-compose up -d
```

```
# View logs
```

```
docker-compose logs -f
```

```
# Stop services
```

```
docker-compose down
```

3. Access the Application

- **Frontend:** <http://localhost:5173>
- **Backend API:** <http://localhost:8001>
- **API Documentation:** <http://localhost:8001/docs> (Swagger UI)
- **Database:** localhost:5433

Project Structure

```
Docker compose/
├── docker-compose.yml      # Docker Compose configuration
├── README.md              # This file
├──
├── frontend/              # Vue.js 3 Frontend
│   ├── Dockerfile         # Node.js container
│   └── configuration
│       ├── package.json   # Dependencies
│       ├── vite.config.js # Vite configuration
│       ├── tailwind.config.js # Tailwind CSS configuration
│       ├── postcss.config.js # PostCSS configuration
│       ├── index.html     # HTML entry point
│       └── src/
│           ├── main.js    # Vue app entry
│           ├── App.vue    # Main component
│           ├── style.css  # Global styles
│           └── utils/
│               └── topology.js # Helper functions
```

```
|
| └─ backend/                                # Python FastAPI Backend
|   └─ Dockerfile                            # Python container configuration
|   └─ requirements.txt                      # Python dependencies
|   └─ main.py                              # FastAPI application
|   └─ .env                                 # Environment variables
(optional)
└─ database/                                # PostgreSQL Database
    └─ init.sql                             # Database schema initialization
```

Configuration

Environment Variables

Backend (.env)

```
DATABASE_URL=postgresql://topology_user:topology_password@database:5432/topology_db
PYTHONUNBUFFERED=1
```

Docker Compose

```
# Database Credentials
POSTGRES_DB: topology_db
POSTGRES_USER: topology_user
POSTGRES_PASSWORD: topology_password

# API URL (for Frontend – browser เรียกผ่าน host port)
VITE_API_URL: http://localhost:8001
```

Database Schema

hosts table

```
CREATE TABLE hosts (  
  id SERIAL PRIMARY KEY,  
  ip_address VARCHAR(15) UNIQUE,  
  mac_address VARCHAR(17),  
  hostname VARCHAR(255),  
  device_type VARCHAR(50),  
  glyph VARCHAR(10),  
  is_gateway BOOLEAN,  
  is_self BOOLEAN,  
  online BOOLEAN,  
  last_seen TIMESTAMP,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);
```

action_history table

```
CREATE TABLE action_history (  
  id SERIAL PRIMARY KEY,  
  host_ip VARCHAR(15),  
  action_type VARCHAR(50), -- 'message', 'ping', 'wol'  
  action_data JSONB,       -- Request data  
  result JSONB,            -- Response data  
  executed_at TIMESTAMP  
);
```

API Endpoints

Docker Topology

GET /api/topology

- อ่าน Docker daemon (ผ่าน socket ที่ mount เข้ามา) แล้วคืนกราฟ Machine → Container → Image
- Response:

```
{  
  "machine": {
```

```

    "id": "machine",
    "name": "docker-desktop",
    "os": "Docker Desktop",
    "containers": 53,
    "images": 33,
    "ncpu": 8,
    "mem": 8259776512
  },
  "containers": [
    {
      "id": "c-1a2b3c4d5e6f",
      "name": "topology_backend",
      "status": "running",
      "image": "dockercompose-backend:latest",
      "ports": "8001->8000/tcp"
    }
  ],
  "images": [
    { "id": "img-9f8e7d6c5b4a", "name": "postgres:16-alpine",
"size": 273000000, "tags": ["postgres:16-alpine"] }
  ],
  "edges": [
    { "from": "machine", "to": "c-1a2b3c4d5e6f", "kind": "runs"
},
    { "from": "c-1a2b3c4d5e6f", "to": "img-9f8e7d6c5b4a",
"kind": "image" }
  ]
}

```

Network Discovery

GET /api/hosts

- Discovers all devices on the network
- Returns: List of hosts with IP, MAC, hostname, status
- Response:

```

{
  "self": "192.168.1.100",
  "gateway": "192.168.1.1",
  "subnet": "192.168.1.0/24",

```



```
"count": 15,
"hosts": [
  {
    "ip": "192.168.1.1",
    "mac": "aa:bb:cc:dd:ee:ff",
    "name": "router.local",
    "isSelf": false,
    "isGateway": true,
    "online": true,
    "glyph": "🌐"
  }
]
}
```

Actions

POST /api/action

- Execute action on target host
- Request:

```
{
  "ip": "192.168.1.50",
  "mac": "aa:bb:cc:dd:ee:ff",
  "type": "message|ping|wol",
  "text": "Hello from topology",
  "via": "agent|psexec"
}
```

- Response:

```
{
  "ok": true,
  "via": "agent|ping|wol",
  "latency": 5.2,
  "host": "target-machine"
}
```

History

GET /api/history

- Get recent action history
- Returns: Array of executed actions with results

GET /api/hosts/{ip}

- Get details for specific host
 - Returns: Host information
-

Features

Frontend Features

-  **Docker Topology View:** วาด Machine → Container → Image เป็น 3 ชั้น (ค่าเริ่มต้น)
-  **Beautiful UI:** Dark theme with gradient backgrounds
-  **Interactive Nodes:** Click to select, drag to move (Pointer Events)
-  **Real-time Network Scanning:** Discover hosts automatically
-  **Live Updates:** Simulate device status changes
-  **Action Panel:** Send messages, ping, or wake devices
-  **Preset Modes:** Docker / Sample / Degraded / Live scenarios

Backend Features

-  **Docker Topology:** อ่าน Machine/Container/Image จริงผ่าน Docker SDK + socket
-  **Network Discovery:** ARP scanning + reverse DNS
-  **Ping Sweep:** Concurrent host reachability check
-  **Database Persistence:** Store scan results and action history
-  **Action History:** Track all sent messages and commands
-  **Agent Integration:** Support for topology-agent receivers

Database Features

-  **Historical Data:** Track network changes over time
-  **Data Persistence:** PostgreSQL with proper indices

-  **Analytics Ready:** Structured data for monitoring
-

Docker Commands

```
# Start all services
docker-compose up -d

# View logs
docker-compose logs -f backend
docker-compose logs -f frontend
docker-compose logs -f database

# Stop services
docker-compose down

# Stop and remove volumes
docker-compose down -v

# Rebuild images
docker-compose build --no-cache

# Restart a specific service
docker-compose restart backend

# Execute command in container
docker-compose exec backend python -c "import sqlalchemy;
print(sqlalchemy.__version__)"

# View running containers
docker-compose ps
```

Network Scanning Logic (ported from agent.js)

Flow:

1. **Get Active Network:** Find default route interface
2. **Ping Sweep:** Concurrent ICMP to all IPs in subnet

3. **ARP Table Parsing:** Extract IP-MAC mappings
4. **Reverse DNS:** Resolve hostnames
5. **Device Classification:** Categorize device type
6. **Database Save:** Persist results

Example (Python FastAPI):

```
async def discover_network(db: Session):
    net = await get_active_network() # Get default interface
    subnet_base = ".".join(net["self"].split(".")[0:3])

    # Ping sweep (concurrent)
    for i in range(1, 255):
        asyncio.create_task(ping_host(f"{subnet_base}.{i}"))

    # Parse ARP table
    arp_hosts = await get_arp_table(subnet_base)

    # Resolve names and save
    for host in arp_hosts:
        host_info = Host(ip_address=host["ip"], ...)
        db.add(host_info)

    return hosts
```

Integration with agent.js

The **topology-agent** (original agent.js) can still be used:

```
# On target machine, run:
node agent.js

# Agent listens on :9911 and receives messages from backend
# GET /info          → Returns host information
# POST /message      → Displays popup message
```

Backend integration (Python FastAPI):

```
async def send_message_to_agent(ip: str, text: str):
    async with httpx.AsyncClient(timeout=1.5) as client:
        response = await client.post(
            f"http://{ip}:9911/message",
            json={"text": text, "from": "topology-server"}
        )
    return response.json()
```

Technology Stack Summary

Component	Technology	Version
Frontend	Vue.js	3.3+
	Tailwind CSS	3.3+
	Vite	5.0+
	Axios	1.6+
Backend	Python	3.11
	FastAPI	0.104+
	SQLAlchemy	2.0+
	Docker SDK	7.1
	Scapy	2.5+
Database	PostgreSQL	16
Container	Docker	Latest
	Docker Compose	v2

Troubleshooting

Frontend not connecting to Backend

```
# Check backend is running
curl http://localhost:8001
```

```
# Check API URL in frontend env
# VITE_API_URL should be http://localhost:8001 (browser เรียกผ่าน host port)
```

Docker Topology ว่างเปล่า / 500

```
# ตรวจสอบ backend ผ่าน docker.sock ได้
curl http://localhost:8001/api/topology

# ต้อง mount socket ใน docker-compose.yml:
#   - /var/run/docker.sock:/var/run/docker.sock:ro
# ถ้าเพิ่งเพิ่ม docker SDK ต้อง rebuild:
docker compose up -d --build backend
```

Database connection failed

```
# Check PostgreSQL container is running
docker-compose ps

# Check logs
docker-compose logs database

# Verify credentials in docker-compose.yml
```

Network discovery not working

```
# Check backend logs
docker-compose logs backend

# Test network access
docker-compose exec backend ping 8.8.8.8

# Verify interface detection
docker-compose exec backend ip route show
```

Building issues

```
# Clear cache and rebuild
docker-compose down -v
```

```
docker-compose build --no-cache
docker-compose up -d
```



Development

Running locally (without Docker)

Frontend:

```
cd frontend
npm install
npm run dev # http://localhost:5173
```

Backend:

```
cd backend
pip install -r requirements.txt
python main.py # http://localhost:8000
```

Database:

```
# PostgreSQL must be running locally
# Update DATABASE_URL in main.py
```



References

- [Vue.js 3 Documentation](#)
 - [Tailwind CSS Documentation](#)
 - [FastAPI Documentation](#)
 - [Docker Compose Documentation](#)
 - [PostgreSQL Documentation](#)
-



Course Information

- **Course:** CT519 - การประมวลผลคลาวด์ (Cloud Computing)
 - **Assignment:** Docker Compose Project
 - **Requirements:**
 - ✓ 3 Containers (Frontend, Backend, Database)
 - ✓ Tech Stack: Vue.js3, PostgreSQL, Python FastAPI, Tailwind
 - ✓ Network Scanning: Ported logic from agent.js/server.js
 - ✓ Use Case: Live System Topology visualization
-



Support

For issues or questions:

1. Check logs: `docker-compose logs -f`
 2. Verify configuration: Check `docker-compose.yml`
 3. Test APIs: Open <http://localhost:8001/docs>
-

Last Updated: June 28, 2026

Version: 1.0.0